

Python Complete Notes

From Basics to Advanced - Explanation, Syntax and Example for Every Concept

- [Table of Contents](#)
- [1. Python Basics - Variables and Data Types](#)
- [2. Primitive Data Types](#)
- [3. Non-Primitive Data Types](#)
- [4. General Built-in Functions](#)
- [5. String Built-in Methods](#)
- [6. Membership and Identity Operators](#)
- [7. Object-Oriented Programming \(OOPs\)](#)
- [8. Encapsulation and Bean Class](#)
- [9. Inheritance \(is-a Relation\)](#)
- [10. Polymorphism](#)
- [11. Abstraction](#)
- [12. Singleton Class](#)
- [13. Functions \(4 Types\) and Inner Functions](#)
- [14. Decorators](#)
- [15. File Handling](#)
- [16. Exception Handling](#)
- [17. Higher Order Functions](#)
- [18. Iterators](#)
- [19. Generators \(Function and Expression\)](#)
- [20. Packing and Unpacking](#)
- [21. Comprehensions](#)
- [22. Database Connection \(SQLite\)](#)
- [23. JSON and Pickle \(Serialization and Deserialization\)](#)
- [24. Regular Expressions \(re module\)](#)
- [25. Additional Important Concepts](#)
- [End of Notes](#)

Table of Contents

1. Python Basics - Variables and Data Types
2. Primitive Data Types
3. Non-Primitive Data Types
4. General Built-in Functions
5. String Built-in Methods
6. Membership and Identity Operators
7. Object-Oriented Programming (OOPs)
8. Encapsulation and Bean Class
9. Inheritance (is-a Relation)
10. Has-a Relation (Aggregation and Composition)
11. Polymorphism
12. Abstraction
13. Singleton Class
14. Functions (4 Types) and Inner Functions
15. Decorators
16. File Handling
17. Exception Handling
18. Higher Order Functions
19. Iterators
20. Generators (Function and Expression)
21. Packing and Unpacking
22. Comprehensions
23. Database Connection
24. JSON and Pickle (Serialization and Deserialization)
25. Regular Expressions
26. Additional Important Concepts

1. Python Basics - Variables and Data Types

Explanation: Python is a high-level, interpreted, general-purpose programming language. It supports multiple programming paradigms including procedural, object-oriented, and functional programming, and is known for its simple, readable syntax.

1.1 Variables

Explanation: A variable is a named location in memory used to store data. Python variables are dynamically typed, meaning you do not need to declare the type explicitly; it is decided automatically based on the assigned value.

Syntax:

```
variable_name = value
```

Example:

```
name = "Alice"      # str
age = 25             # int
height = 5.7         # float
is_student = True    # bool
```

Variable Naming Rules: A variable name must start with a letter or an underscore. It can contain letters, numbers, and underscores, but cannot be a Python keyword such as `if`, `for`, or `class`. Variable names are case-sensitive, meaning `name`, `Name`, and `NAME` are all different variables. It is conventional to use `snake_case` for variable and function names.

1.2 Multiple Assignment

Explanation: Python allows assigning values to multiple variables in a single line, assigning the same value to multiple variables, and swapping variable values without a temporary variable.

Syntax:

```
a, b, c = value1, value2, value3
x = y = z = same_value
a, b = b, a
```

Example:

```
a, b, c = 1, 2, 3      # Multiple values
x = y = z = 0          # Same value
a, b = b, a            # Swap values
print(a, b, c)
```

1.3 Type Casting

Explanation: Type casting is the process of converting a value from one data type to another using built-in conversion functions.

Syntax:

```
int(value)
float(value)
str(value)
bool(value)
```

Example:

```
x = int("10")          # String to int: 10
y = float("3.14")       # String to float: 3.14
z = str(100)            # Int to string: "100"
a = bool(1)             # Int to bool: True
b = int(3.7)            # Float to int: 3 (truncates decimal)
```

1.4 Data Type Categories

Explanation: Python data types fall into two broad categories: Primitive (Fundamental) Data Types, which store a single simple value, and Non-Primitive (Derived) Data Types, which store a collection of values.

2. Primitive Data Types

Explanation: Primitive data types are the basic built-in types provided by Python that hold only one simple value at a time.

2.1 int (Integer)

Explanation: The int type represents whole numbers, positive or negative, without a decimal point. Python supports arbitrary-precision integers, meaning integers can be as large as memory allows. Integers can also be written in binary, octal, and hexadecimal form.

Syntax:

```
variable_name = whole_number
variable_name = 0b_binary_digits
variable_name = 0o_octal_digits
variable_name = 0x_hex_digits
```

Example:

```
a = 25
b = -10
large = 10 ** 100    # Very large number
binary = 0b1010      # Binary: 10
octal = 0o17         # Octal: 15
hexa = 0xFF          # Hex: 255
print(type(a))       # Output: <class 'int'>
```

Useful int methods: `bit_length()` returns the number of bits needed to represent the integer; `bit_count()` returns the number of set bits (1s); `to_bytes()` converts the integer to bytes; `from_bytes()` converts bytes back to an integer.

2.2 float (Floating Point)

Explanation: The float type represents numbers with a decimal point, using IEEE 754 double-precision internally. Floats can also be written using scientific notation.

Syntax:

```
variable_name = number.decimal
variable_name = mantissa_e_exponent
```

Example:

```
pi = 3.14159
scientific = 2.5e3    # 2500.0
scientific2 = 1.5e-4  # 0.00015
print(type(pi))      # Output: <class 'float'>
```

Useful float methods: `is_integer()` returns True if the float has no fractional part; `as_integer_ratio()` returns a tuple representing the float as a fraction.

2.3 complex (Complex Number)

Explanation: The complex type represents complex numbers, which have a real part and an imaginary part. The imaginary part is written with a suffix "j".

Syntax:

```
variable_name = real_part + imaginary_partj
variable_name = complex(real, imaginary)
```

Example:

```
z1 = 3 + 4j
```

```
z2 = complex(2, 5)      # (2+5j)
print(z1.real)          # Output: 3.0
print(z1.imag)          # Output: 4.0
print(abs(z1))          # Output: 5.0 (magnitude)
```

2.4 bool (Boolean)

Explanation: The bool type represents one of two values, True or False. In Python, bool is technically a subclass of int, where True behaves as 1 and False behaves as 0.

Syntax:

```
variable_name = True_or_False
```

Example:

```
x = True
y = False
print(type(x))      # Output: <class 'bool'>
print(True + True)  # Output: 2
print(True * 5)     # Output: 5
```

2.5 str (String)

Explanation: The str type represents a sequence of characters enclosed within single quotes, double quotes, or triple quotes. Strings are immutable, meaning once created they cannot be changed in place. A string prefixed with “r” is a raw string, where escape sequences are not processed.

Syntax:

```
variable_name = "sequence of characters"
variable_name = '''triple quoted string'''
variable_name = r"raw string"
```

Example:

```
s1 = "Hello"
s2 = 'World'
s3 = """Multi
line string"""
s4 = r"Raw \nstring"  # Escape sequence not processed
```

String Indexing and Slicing: Strings support indexing (accessing a single character by position) and slicing (extracting a portion of the string using start, end, and step values).

Syntax:

```
string[index]
string[start:end:step]
```

Example:

```
s = "Hello World"
print(s[0])      # Output: H
print(s[-1])     # Output: d
print(s[0:5])    # Output: Hello
print(s[6:])     # Output: World
print(s[::-1])   # Output: dlrow olleH (reversed string)
```

2.6 NoneType

Explanation: NoneType represents the absence of a value. It has only one possible value, None, and is commonly used to indicate that a variable has no value assigned yet.

Syntax:

```
variable_name = None
```

Example:

```
x = None
print(type(x))      # Output: <class 'NoneType'>
print(x is None)    # Output: True
```

3. Non-Primitive Data Types

Explanation: Non-primitive data types can hold multiple values or a collection of values, unlike primitive types which hold only one value.

3.1 List

Explanation: A list is an ordered, mutable (changeable) collection of items that allows duplicate elements and can hold mixed data types. Lists are defined using square brackets.

Syntax:

```
variable_name = [item1, item2, item3]
```

Example:

```
fruits = ["apple", "banana", "cherry"]
mixed = [1, "two", 3.0, True, None]
fruits[0] = "orange"
print(fruits)  # Output: ['orange', 'banana', 'cherry']
```

List Built-in Methods

append(item)

Explanation: Adds a single item to the end of the list.

Syntax: `list.append(item)`

Example:

```
lst = [1, 2, 3]
lst.append(4)
print(lst)  # Output: [1, 2, 3, 4]
```

insert(index, item)

Explanation: Inserts an item at a specific index, shifting the remaining elements to the right.

Syntax: `list.insert(index, item)`

Example:

```
lst = [1, 2, 3]
lst.insert(1, 5)
print(lst)  # Output: [1, 5, 2, 3]
```

extend(iterable)

Explanation: Adds all the elements of another iterable to the end of the current list.

Syntax: `list.extend(iterable)`

Example:

```
lst = [1, 2]
lst.extend([3, 4])
print(lst)  # Output: [1, 2, 3, 4]
```

remove(item)

Explanation: Removes the first occurrence of the specified value. Raises a `ValueError` if the value is not found.

Syntax: `list.remove(item)`

Example:

```
lst = [1, 2, 3, 2]
lst.remove(2)
print(lst)    # Output: [1, 3, 2]
```

pop(index)

Explanation: Removes and returns the item at the given index. If no index is given, it removes and returns the last item.

Syntax: `list.pop(index)`

Example:

```
lst = [1, 2, 3]
value = lst.pop()
print(value)    # Output: 3
print(lst)      # Output: [1, 2]
```

clear()

Explanation: Removes all elements from the list, leaving it empty.

Syntax: `list.clear()`

Example:

```
lst = [1, 2, 3]
lst.clear()
print(lst)      # Output: []
```

index(item)

Explanation: Returns the index of the first occurrence of the specified value.

Syntax: `list.index(item)`

Example:

```
lst = [1, 2, 3]
print(lst.index(3))    # Output: 2
```

count(item)

Explanation: Returns the number of times the specified value appears in the list.

Syntax: `list.count(item)`

Example:

```
lst = [1, 2, 2, 3]
print(lst.count(2))    # Output: 2
```

sort()

Explanation: Sorts the list in ascending order, in place (modifying the original list).

Syntax: `list.sort()`

Example:

```
lst = [3, 1, 2]
lst.sort()
print(lst)    # Output: [1, 2, 3]
```

sorted(list)

Explanation: Returns a new sorted list without modifying the original list. This is a built-in function, not a method.

Syntax: `sorted(list)`

Example:

```
lst = [3, 1, 2]
print(sorted(lst))    # Output: [1, 2, 3]
print(lst)             # Output: [3, 1, 2] (unchanged)
```

reverse()

Explanation: Reverses the order of elements in the list, in place.

Syntax: `list.reverse()`

Example:

```
lst = [1, 2, 3]
lst.reverse()
print(lst)    # Output: [3, 2, 1]
```

copy()

Explanation: Returns a shallow copy of the list.

Syntax: `list.copy()`

Example:

```
lst = [1, 2, 3]
new_lst = lst.copy()
print(new_lst)    # Output: [1, 2, 3]
```

len(list)

Explanation: Returns the total number of elements present in the list.

Syntax: `len(list)`

Example:

```
lst = [1, 2, 3]
print(len(lst))    # Output: 3
```

3.2 Tuple

Explanation: A tuple is an ordered, immutable collection of items that allows duplicate elements. Since tuples cannot be changed after creation, they are faster than lists and can be used as dictionary keys. Tuples are defined using parentheses. A single-element tuple requires a trailing comma.

Syntax:

```
variable_name = (item1, item2, item3)
single_element_tuple = (item1,)
```

Example:

```
colors = ("red", "green", "blue")
numbers = (1, 2, 3, 4, 5)
single = (42,)    # Single element tuple, note the comma
print(colors)
```

Tuple Built-in Methods

count(item)

Explanation: Returns the number of times the value appears in the tuple.

Syntax: `tuple.count(item)`

Example:

```
t = (1, 2, 2, 3)
print(t.count(2))    # Output: 2
```

index(item)

Explanation: Returns the index of the first occurrence of the value in the tuple.

Syntax: `tuple.index(item)`

Example:

```
t = (1, 2, 3)
print(t.index(2))    # Output: 1
```

len(tuple)

Explanation: Returns the total number of elements in the tuple.

Syntax: `len(tuple)`

Example:

```
t = (1, 2, 3)
print(len(t)) # Output: 3
```

3.3 Dictionary

Explanation: A dictionary is an insertion-ordered (Python 3.7+), mutable collection that stores data as key-value pairs. Keys must be unique and immutable. Dictionaries are defined using curly braces.

Syntax:

```
variable_name = {key1: value1, key2: value2}
```

Example:

```
student = {
    "name": "Alice",
    "age": 22,
    "grades": [90, 85, 92]
}
print(student["name"]) # Output: Alice
```

Dictionary Built-in Methods

get(key, default)

Explanation: Returns the value for the given key. If the key does not exist, it returns None, or the specified default value instead of raising an error.

Syntax: dict.get(key, default)

Example:

```
d = {"name": "Alice"}
print(d.get("age")) # Output: None
print(d.get("age", 0)) # Output: 0
```

keys()

Explanation: Returns a view object containing all the keys of the dictionary.

Syntax: dict.keys()

Example:

```
d = {"a": 1, "b": 2}
print(d.keys()) # Output: dict_keys(['a', 'b'])
```

values()

Explanation: Returns a view object containing all the values of the dictionary.

Syntax: dict.values()

Example:

```
d = {"a": 1, "b": 2}
print(d.values()) # Output: dict_values([1, 2])
```

items()

Explanation: Returns a view object containing all key-value pairs as tuples.

Syntax: dict.items()

Example:

```
d = {"a": 1, "b": 2}
for key, val in d.items():
    print(key, val)
```

update(dict2)

Explanation: Updates the dictionary by merging in key-value pairs from another dictionary, adding new keys and overwriting existing ones.

Syntax: `dict.update(dict2)`

Example:

```
d = {"name": "Alice"}
d.update({"gpa": 3.8})
print(d) # Output: {'name': 'Alice', 'gpa': 3.8}
```

pop(key)

Explanation: Removes the specified key from the dictionary and returns its value.

Syntax: `dict.pop(key)`

Example:

```
d = {"name": "Alice", "age": 22}
val = d.pop("age")
print(val) # Output: 22
```

popitem()

Explanation: Removes and returns the last inserted key-value pair as a tuple.

Syntax: `dict.popitem()`

Example:

```
d = {"a": 1, "b": 2}
print(d.popitem()) # Output: ('b', 2)
```

setdefault(key, default)

Explanation: Returns the value for the given key if it exists; otherwise it inserts the key with the given default value.

Syntax: `dict.setdefault(key, default)`

Example:

```
d = {"name": "Alice"}
d.setdefault("age", 0)
print(d) # Output: {'name': 'Alice', 'age': 0}
```

copy()

Explanation: Returns a shallow copy of the dictionary.

Syntax: `dict.copy()`

Example:

```
d = {"a": 1}
d2 = d.copy()
print(d2) # Output: {'a': 1}
```

clear()

Explanation: Removes all key-value pairs from the dictionary, making it empty.

Syntax: `dict.clear()`

Example:

```
d = {"a": 1}
d.clear()
print(d) # Output: {}
```

fromkeys(keys, value)

Explanation: Creates a new dictionary using elements of a given sequence as keys, all set to the same specified value.

Syntax: `dict.fromkeys(keys, value)`

Example:

```
keys = ["a", "b"]
d = dict.fromkeys(keys, 0)
print(d) # Output: {'a': 0, 'b': 0}
```

len(dict)

Explanation: Returns the total number of key-value pairs in the dictionary.

Syntax: `len(dict)`

Example:

```
d = {"a": 1, "b": 2}
print(len(d))    # Output: 2
```

3.4 Set

Explanation: A set is an unordered, mutable collection of unique elements; it does not allow duplicate values. Sets are defined using curly braces or the `set()` function. An empty set must be created with `set()`, since `{}` creates an empty dictionary instead.

Syntax:

```
variable_name = {item1, item2, item3}
empty_set = set()
```

Example:

```
fruits = {"apple", "banana", "cherry"}
numbers = {1, 2, 3, 4, 5}
print(fruits)
```

Set Built-in Methods

`add(item)`

Explanation: Adds a single element to the set.

Syntax: `set.add(item)`

Example:

```
s = {1, 2, 3}
s.add(4)
print(s)    # Output: {1, 2, 3, 4}
```

`update(iterable)`

Explanation: Adds multiple elements from an iterable to the set at once.

Syntax: `set.update(iterable)`

Example:

```
s = {1, 2}
s.update([3, 4])
print(s)    # Output: {1, 2, 3, 4}
```

`remove(item)`

Explanation: Removes the specified element from the set. Raises a `KeyError` if the element is not present.

Syntax: `set.remove(item)`

Example:

```
s = {1, 2, 3}
s.remove(2)
print(s)    # Output: {1, 3}
```

`discard(item)`

Explanation: Removes the specified element if present, but does not raise an error if the element is missing.

Syntax: `set.discard(item)`

Example:

```
s = {1, 2, 3}
s.discard(5)
print(s)    # Output: {1, 2, 3}
```

`pop()`

Explanation: Removes and returns an arbitrary element from the set, since sets are unordered.

Syntax: `set.pop()`

Example:

```
s = {1, 2, 3}
print(s.pop()) # Output: an arbitrary element, e.g. 1
```

clear()

Explanation: Removes all elements from the set, making it empty.

Syntax: `set.clear()`

Example:

```
s = {1, 2, 3}
s.clear()
print(s) # Output: set()
```

union(set2) or the | operator

Explanation: Returns a new set containing all elements from both sets, without duplicates.

Syntax: `set.union(set2)` or `set | set2`

Example:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b) # Output: {1, 2, 3, 4, 5, 6}
```

intersection(set2) or the & operator

Explanation: Returns a new set containing only the elements common to both sets.

Syntax: `set.intersection(set2)` or `set & set2`

Example:

```
print(a & b) # Output: {3, 4}
```

difference(set2) or the - operator

Explanation: Returns a new set containing elements present in the first set but not the second.

Syntax: `set.difference(set2)` or `set - set2`

Example:

```
print(a - b) # Output: {1, 2}
```

symmetric_difference(set2) or the ^ operator

Explanation: Returns a new set containing elements present in either set but not in both.

Syntax: `set.symmetric_difference(set2)` or `set ^ set2`

Example:

```
print(a ^ b) # Output: {1, 2, 5, 6}
```

issubset(set2)

Explanation: Returns True if all elements of the current set are present in set2.

Syntax: `set.issubset(set2)`

Example:

```
a = {1, 2}
b = {1, 2, 3}
print(a.issubset(b)) # Output: True
```

issuperset(set2)

Explanation: Returns True if the current set contains all elements of set2.

Syntax: `set.issuperset(set2)`

Example:

```
print(b.issuperset(a)) # Output: True
```

copy()

Explanation: Returns a shallow copy of the set.

Syntax: `set.copy()`

Example:

```
s = {1, 2, 3}
s2 = s.copy()
print(s2) # Output: {1, 2, 3}
```

len(set)

Explanation: Returns the total number of elements present in the set.

Syntax: `len(set)`

Example:

```
s = {1, 2, 3}
print(len(s)) # Output: 3
```

3.5 frozenset

Explanation: A frozenset is the immutable version of a set. Since it cannot be changed after creation, it can be used as a dictionary key or as an element inside another set (regular sets cannot be used this way).

Syntax:

```
variable_name = frozenset(iterable)
```

Example:

```
fs = frozenset([1, 2, 3, 4])
print(fs)
# fs.add(5) # This would raise AttributeError, frozensets are immutable
```

3.6 bytes and bytearray

Explanation: bytes is an immutable sequence of byte values, while bytearray is its mutable counterpart. Both are used for handling binary data.

Syntax:

```
bytes(size_or_source)
bytearray(source)
```

Example:

```
b = bytes(5) # b'\x00\x00\x00\x00\x00'
ba = bytearray(b"hello")
ba[0] = ord("H") # Modify bytearray in place
print(ba) # Output: bytearray(b'Hello')
```

4. General Built-in Functions

Explanation: Python provides many built-in functions that are always available without importing any module. They are grouped below by purpose, with an explanation, syntax and example for each.

4.1 Type Conversion Functions

int(value) — Explanation: Converts a value (string or number) into an integer. Syntax: `int(value)`. Example: `int("10")` returns `10`.

float(value) — Explanation: Converts a value into a floating-point number. Syntax: `float(value)`. Example: `float("3.14")` returns `3.14`.

str(value) — Explanation: Converts a value into its string representation. Syntax: `str(value)`. Example: `str(100)` returns `"100"`.

bool(value) — Explanation: Converts a value into a Boolean, based on its truthiness. Syntax: `bool(value)`. Example: `bool(0)` returns `False`.

list(iterable) — Explanation: Converts an iterable into a list. Syntax: `list(iterable)`. Example: `list("abc")` returns `['a', 'b', 'c']`.

tuple(iterable) — Explanation: Converts an iterable into a tuple. Syntax: `tuple(iterable)`. Example: `tuple([1, 2])` returns `(1, 2)`.

set(iterable) — Explanation: Converts an iterable into a set, removing duplicates. Syntax: `set(iterable)`. Example: `set([1, 1, 2])` returns `{1, 2}`.

dict(iterable) — Explanation: Converts a sequence of key-value pairs into a dictionary. Syntax: `dict(iterable)`. Example: `dict(['a', 1])` returns `{'a': 1}`.

complex(real, imag) — Explanation: Creates a complex number. Syntax: `complex(real, imag)`. Example: `complex(1, 2)` returns `(1+2j)`.

bytes(string, encoding) — Explanation: Converts a string into bytes using a specified encoding. Syntax: `bytes(string, encoding)`. Example: `bytes("hi", "utf-8")` returns `b'hi'`.

4.2 Math Functions

abs(x) — Explanation: Returns the absolute (positive) value of a number. Example: `abs(-5)` returns `5`.

round(x, n) — Explanation: Rounds a number to n decimal places. Example: `round(3.14159, 2)` returns `3.14`.

pow(base, exp, mod) — Explanation: Returns base raised to exp; if mod is provided, returns the result modulo mod. Example: `pow(2, 3)` returns `8`; `pow(2, 3, 5)` returns `3`.

divmod(a, b) — Explanation: Returns a tuple containing the quotient and remainder of a divided by b. Example: `divmod(17, 5)` returns `(3, 2)`.

sum(iterable) — Explanation: Returns the sum of all elements in an iterable. Example: `sum([1, 2, 3, 4])` returns `10`.

max(iterable_or_values) — Explanation: Returns the largest value. Example: `max(1, 2, 3)` returns `3`.

min(iterable_or_values) — Explanation: Returns the smallest value. Example: `min(1, 2, 3)` returns `1`.

4.3 Sequence Functions

len(sequence) — Explanation: Returns the number of items in a sequence. Example: `len([1, 2, 3])` returns `3`.

sorted(iterable) — Explanation: Returns a new sorted list from the items of an iterable. Example: `sorted([3, 1, 2])` returns `[1, 2, 3]`.

reversed(sequence) — Explanation: Returns a reverse iterator over the sequence. Example: `list(reversed([1, 2, 3]))` returns `[3, 2, 1]`.

enumerate(iterable, start) — Explanation: Returns an enumerate object producing pairs of index and value, useful in loops when you need both. Example: `list(enumerate(["a", "b"]))` returns `[(0, 'a'), (1, 'b')]`.

zip(iter1, iter2) — Explanation: Combines multiple iterables element-wise into tuples. Example: `list(zip([1, 2], ["a", "b"]))` returns `[(1, 'a'), (2, 'b')]`.

map(function, iterable) — Explanation: Applies a function to every item of an iterable and returns an iterator of results. Example: `list(map(str, [1, 2, 3]))` returns `['1', '2', '3']`.

filter(function, iterable) — Explanation: Filters items of an iterable for which the function returns True. Example: `list(filter(bool, [0, 1, "", 2]))` returns `[1, 2]`.

4.4 Input and Output Functions

input(prompt) — Explanation: Reads a line of text typed by the user and returns it as a string. Example: `x = input("Enter: ")`.

print(values, sep, end) — Explanation: Prints values to the console; sep controls the separator between values and end controls what is printed at the end (default is a newline). Example: `print("a", "b", sep="-")` prints `a-b`.

4.5 Other Important Functions

type(x) — Explanation: Returns the type of the object x. Example: `type(5)` returns `<class 'int'>`.

isinstance(x, type) — Explanation: Returns True if x is an instance of the given type. Example: `isinstance(5, int)` returns `True`.

id(x) — Explanation: Returns the unique memory address (identity) of the object. Example: `id(x)`.

hash(x) — Explanation: Returns the hash value of an object, used internally for dictionary and set lookups. Example: `hash("abc")`.

dir(x) — Explanation: Returns a list of valid attributes and methods for the object. Example: `dir([1, 2, 3])`.

vars(x) — Explanation: Returns the `__dict__` attribute of an object, showing its writable attributes. Example: `vars(obj)` .

globals() — Explanation: Returns a dictionary representing the current global symbol table.

locals() — Explanation: Returns a dictionary representing the current local symbol table.

eval(expression) — Explanation: Evaluates a string as a Python expression and returns the result. Example: `eval("2+3")` returns `5` .

exec(code) — Explanation: Executes a string of Python code (statements, not just expressions). Example: `exec("x = 5")` .

chr(number) — Explanation: Returns the character represented by a given Unicode code point number. Example: `chr(65)` returns `"A"` .

ord(character) — Explanation: Returns the Unicode code point number of a given character. Example: `ord("A")` returns `65` .

bin(number) — Explanation: Converts an integer into its binary string representation. Example: `bin(10)` returns `"0b1010"` .

oct(number) — Explanation: Converts an integer into its octal string representation. Example: `oct(10)` returns `"0o12"` .

hex(number) — Explanation: Converts an integer into its hexadecimal string representation. Example: `hex(255)` returns `"0xff"` .

range(start, stop, step) — Explanation: Generates a sequence of numbers, commonly used in loops. Example: `list(range(5))` returns `[0, 1, 2, 3, 4]` .

any(iterable) — Explanation: Returns True if at least one element in the iterable is truthy. Example: `any([0, False, None])` returns `False` .

all(iterable) — Explanation: Returns True only if all elements in the iterable are truthy. Example: `all([1, True, "hi"])` returns `True` .

callable(x) — Explanation: Returns True if the object x can be called like a function. Example: `callable(print)` returns `True` .

repr(x) — Explanation: Returns the official, unambiguous string representation of an object, mainly for debugging. Example: `repr("hi")` returns `"'hi'"` .

format(value, spec) — Explanation: Formats a value according to a given format specification. Example: `format(3.14, ".1f")` returns `"3.1"` .

5. String Built-in Methods

Explanation: Since strings are one of the most used data types, Python provides many built-in methods to work with them. Each method is explained individually below with its syntax and an example.

5.1 Case Conversion Methods

upper() — Explanation: Converts all characters to uppercase. Syntax: `string.upper()` . Example: `"hello world".upper()` returns `"HELLO WORLD"` .

lower() — Explanation: Converts all characters to lowercase. Syntax: `string.lower()` . Example: `"HELLO".lower()` returns `"hello"` .

title() — Explanation: Converts the first character of every word to uppercase. Syntax: `string.title()` . Example: `"hello world".title()` returns `"Hello World"` .

capitalize() — Explanation: Converts the first character of the string to uppercase and the rest to lowercase. Syntax: `string.capitalize()` . Example: `"hello".capitalize()` returns `"Hello"` .

swapcase() — Explanation: Swaps uppercase characters to lowercase and vice versa. Syntax: `string.swapcase()` . Example: `"Hi There".swapcase()` returns `"hI tHERE"` .

casefold() — Explanation: Converts the string to lowercase more aggressively than `lower()`, useful for caseless string matching across languages. Syntax: `string.casefold()` . Example: `"HELLO".casefold()` returns `"hello"` .

5.2 Search and Replace Methods

find(sub) — Explanation: Returns the index of the first occurrence of a substring, or -1 if not found. Syntax: `string.find(sub)` . Example: `"hello world".find("world")` returns `6` .

rfind(sub) — Explanation: Returns the index of the last occurrence of a substring, searching from the right, or -1 if not found. Syntax: `string.rfind(sub)` . Example: `"hello world".rfind("l")` returns `9` .

index(sub) — Explanation: Works like `find()`, but raises a `ValueError` if the substring is not found instead of returning -1. Syntax: `string.index(sub)` . Example: `"hello".index("l")` returns `2` .

count(sub) — Explanation: Counts the number of non-overlapping occurrences of a substring. Syntax: `string.count(sub)` . Example: `"hello world".count("l")` returns `3` .

replace(old, new) — Explanation: Replaces all occurrences of a substring with another substring. Syntax: `string.replace(old, new)` . Example: `"hello".replace("l", "x")` returns `"hexxo"` .

startswith(sub) — Explanation: Returns True if the string starts with the given substring. Syntax: `string.startswith(sub)` . Example: `"hello".startswith("he")` returns `True` .

endswith(sub) — Explanation: Returns True if the string ends with the given substring. Syntax: `string.endswith(sub)` . Example: `"hello".endswith("lo")` returns `True` .

5.3 Strip and Split Methods

strip() — Explanation: Removes leading and trailing whitespace (or specified characters) from the string. Syntax: `string.strip()` . Example: `" hello ".strip()` returns `"hello"` .

lstrip() — Explanation: Removes leading (left-side) whitespace or specified characters. Syntax: `string.lstrip()` . Example: `" hello".lstrip()` returns `"hello "` .

rstrip() — Explanation: Removes trailing (right-side) whitespace or specified characters. Syntax: `string.rstrip()` . Example: `" hello".rstrip()` returns `" hello"` .

split(separator) — Explanation: Splits the string into a list of substrings using the given separator; splits on whitespace by default. Syntax: `string.split(separator)` . Example: `"a,b,c".split(",")` returns `['a', 'b', 'c']` .

join(iterable) — Explanation: Joins elements of an iterable into a single string, placing the original string between each element. Syntax: `"separator".join(iterable)` . Example: `" ".join(["a", "b"])` returns `"a b"` .

splitlines() — Explanation: Splits the string at line breaks and returns a list of lines. Syntax: `string.splitlines()` . Example: `"hello\nworld".splitlines()` returns `['hello', 'world']` .

5.4 Check (Validation) Methods

isalpha() — Explanation: Returns True if all characters are alphabets and the string is not empty. Syntax: `string.isalpha()` . Example: `"abc".isalpha()` returns `True` .

isdigit() — Explanation: Returns True if all characters are digits. Syntax: `string.isdigit()` . Example: `"123".isdigit()` returns `True` .

isalnum() — Explanation: Returns True if all characters are alphanumeric (letters and digits only). Syntax: `string.isalnum()` . Example: `"abc123".isalnum()` returns `True` .

islower() — Explanation: Returns True if all cased characters are lowercase. Syntax: `string.islower()` . Example: `"hello".islower()` returns `True` .

isupper() — Explanation: Returns True if all cased characters are uppercase. Syntax: `string.isupper()` . Example: `"HELLO".isupper()` returns `True` .

istitle() — Explanation: Returns True if the string follows title case (each word starts with an uppercase letter). Syntax: `string.istitle()` . Example: `"Hello World".istitle()` returns `True` .

isspace() — Explanation: Returns True if the string contains only whitespace characters. Syntax: `string.isspace()` . Example: `" ".isspace()` returns `True` .

isidentifier() — Explanation: Returns True if the string is a valid Python identifier (variable name). Syntax: `string.isidentifier()` . Example: `"hello".isidentifier()` returns `True` .

5.5 Formatting Methods

format() — Explanation: Formats a string by inserting values into placeholders marked with curly braces. Syntax: `"string { }".format(value)` . Example: `"{ } is { }".format("x", 1)` returns `"x is 1"` .

f-strings — Explanation: A concise way to embed expressions directly inside string literals, prefixed with `"f"`. Syntax: `f"text {variable}"` . Example: `name = "Alice"; f"Hello {name}"` returns `"Hello Alice"` .

Alignment formatting — Explanation: Format specifiers can align text within a given width: `>` for right-align, `<` for left-align, `^` for center-align, and a fill character can be specified before the alignment symbol. Syntax: `"{:>width}".format(value)` . Example: `"{: *^10}".format("hi")` returns `"****hi****"` .

5.6 Encoding Methods

encode(encoding) — Explanation: Converts a string into a bytes object using the specified encoding. Syntax: `string.encode(encoding)` . Example: `"hello".encode("utf-8")` returns `b'hello'` .

decode(encoding) — Explanation: Converts a bytes object back into a string using the specified encoding. Syntax: `bytes_obj.decode(encoding)` . Example: `b"hello".decode("utf-8")` returns `"hello"` .

maketrans() and translate() — Explanation: `maketrans()` creates a translation table mapping characters to replacements; `translate()` applies that table to a string. Syntax: `string.maketrans(from_chars, to_chars)` then `string.translate(table)` . Example: creates a table mapping vowels to numbers and applies it to replace vowels in a string.

5.7 Other String Methods

center(width, fillchar) — Explanation: Centers the string within the given width, padding with the specified fill character on both sides. Syntax: `string.center(width, fillchar)`. Example: `"hello".center(20, "-")` returns a string padded with dashes on both sides.

zfill(width) — Explanation: Pads the string on the left with zeros until it reaches the given width. Syntax: `string.zfill(width)`. Example: `"hello".zfill(10)` returns `"000000hello"`.

removeprefix(prefix) — Explanation: Removes the given prefix from the start of the string if present. Syntax: `string.removeprefix(prefix)`. Example: `"hello123".removeprefix("hello")` returns `"123"`.

removesuffix(suffix) — Explanation: Removes the given suffix from the end of the string if present. Syntax: `string.removesuffix(suffix)`. Example: `"hello123".removesuffix("123")` returns `"hello"`.

expandtabs(tabsize) — Explanation: Replaces tab characters in the string with spaces, based on the given tab size. Syntax: `string.expandtabs(tabsize)`. Example: `"hello".expandtabs(4)`.

len(string) — Explanation: Returns the total number of characters in the string. Syntax: `len(string)`. Example: `len("hello")` returns `5`.

6. Membership and Identity Operators

6.1 Membership Operators

Explanation: Membership operators are used to test whether a value or variable exists within a sequence, such as a list, tuple, string, dictionary, or set. Python provides two types of membership operators.

Types of Membership Operators:

1. `in`
2. `not in`

1. `in` Operator

Explanation: The `"in"` operator checks whether a specified value is present inside a given sequence. It returns `True` if the value is found in the sequence, otherwise it returns `False`. For dictionaries, `"in"` checks the keys by default, not the values; to check values, use `.values()`.

Syntax:

```
value in sequence
```

Example:

```
fruits = ["apple", "banana"]
print("apple" in fruits) # Output: True

student = {"name": "Alice", "age": 22}
print("name" in student) # Output: True (checks keys)
print("Alice" in student) # Output: False
print("Alice" in student.values()) # Output: True
```

2. `not in` Operator

Explanation: The `"not in"` operator checks whether a specified value is absent from a given sequence. It returns `True` if the value is NOT found in the sequence, otherwise it returns `False`.

Syntax:

```
value not in sequence
```

Example:

```
fruits = ["apple", "banana"]
print("mango" not in fruits) # Output: True
```

6.2 Identity Operators

Explanation: Identity operators are used to compare the memory location (identity) of two objects, rather than comparing their values. Python provides two types of identity operators.

Types of Identity Operators:

1. is
2. is not

1. is Operator

Explanation: The “is” operator checks whether two variables point to the exact same object in memory. It returns True if both variables refer to the same object, otherwise it returns False. Note that Python caches small integers (from -5 to 256); comparing two such integers with “is” may return True even without explicit assignment from one to another.

Syntax:

```
variable1 is variable2
```

Example:

```
a = [1, 2, 3]
c = a
print(a is c)    # Output: True, since c refers to the same object as a

x = 256
y = 256
print(x is y)    # Output: True (small integers are cached)

x = 257
y = 257
print(x is y)    # Output: False (outside the cache range)
```

2. is not Operator

Explanation: The “is not” operator checks whether two variables point to different objects in memory. It returns True if the variables refer to different objects, otherwise it returns False.

Syntax:

```
variable1 is not variable2
```

Example:

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a is not b) # Output: True, since a and b are different objects even though values are equal
```

is vs ==: The == operator checks if the values of two variables are equal (it calls the __eq__ method internally), while is checks whether both variables refer to the exact same object in memory. It is a common convention to use is None and is not None rather than == None.

7. Object-Oriented Programming (OOPs)

Explanation: OOPs is a programming paradigm based on the concept of “objects”, where each object bundles data (attributes) and behavior (methods) together. It helps organize code into reusable, real-world-like structures.

Why OOPs is used: Object-Oriented Programming provides code reusability, since code written once can be reused many times. It provides data hiding, protecting the internal state of an object. It provides modularity, since components are independent and interchangeable. It provides scalability, making it easier to extend and maintain large applications. It also helps in real-world modeling, since objects mirror real-world entities naturally.

7.1 Class and Object

Explanation: A class is a blueprint or template that defines the structure and behavior of objects. An object is a specific instance created from that class.

Syntax:

```
class ClassName:
    def __init__(self, parameters):
        self.attribute = parameters

    def method_name(self):
        # method body
```

```
pass
```

Example:

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand    # Instance variable
        self.color = color

    def drive(self):          # Instance method
        return f"{self.brand} is driving"

c1 = Car("Toyota", "Red")
c2 = Car("Honda", "Blue")
print(c1.drive())           # Output: Toyota is driving
```

7.2 The init Method (Constructor)

Explanation: The `__init__` method is the constructor of a class, automatically called whenever a new object is created, used to initialize the object's attributes.

Syntax:

```
def __init__(self, parameters):
    self.attribute = parameters
```

Example:

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

s = Student("Alice", 20, "A")
```

7.3 Instance, Class, and Static Variables/Methods

Explanation: An instance variable belongs to a specific object and is defined inside `__init__` using `self`. A class variable is shared across all instances of the class and is defined directly inside the class body. A static method does not access instance or class data and is defined using the `@staticmethod` decorator. A class method operates on the class itself rather than an instance, and is defined using the `@classmethod` decorator, receiving `cls` as its first parameter.

Syntax:

```
class ClassName:
    class_var = value          # Class variable

    def __init__(self):
        self.inst_var = value  # Instance variable

    @staticmethod
    def static_method():
        pass

    @classmethod
    def class_method(cls):
        return cls.class_var
```

Example:

```
class Example:
    class_var = "I belong to class"

    def __init__(self):
        self.inst_var = "I belong to instance"

    @staticmethod
    def static_method():
        return "No access to class or instance"
```

```
@classmethod
def class_method(cls):
    return cls.class_var
```

7.4 The self Parameter

Explanation: self refers to the current instance of the class. It must be the first parameter of any instance method, but it is never passed explicitly when calling the method - Python passes it automatically.

7.5 The 4 Pillars of OOPs

Explanation: Object-Oriented Programming is built on four foundational principles: 1. Encapsulation - bundling data and hiding it from outside access. 2. Inheritance - reusing code by acquiring properties from a parent class. 3. Polymorphism - allowing the same interface to behave differently. 4. Abstraction - hiding complexity and showing only essential features.

8. Encapsulation and Bean Class

8.1 Encapsulation

Explanation: Encapsulation is the bundling of data (attributes) and the methods that operate on that data within a single unit (a class), while restricting direct access to some of its components using access modifiers.

Access Modifiers in Python: A **public** member (written as `self.name`) is accessible from anywhere. A **protected** member (written with a single underscore prefix, `self._name`) is accessible within the class and its subclasses, by convention. A **private** member (written with a double underscore prefix, `self.__name`) is accessible only within the class itself, since Python applies name mangling to it.

Syntax:

```
class ClassName:
    def __init__(self):
        self.public_var = value
        self._protected_var = value
        self.__private_var = value

    def get_private(self):    # Getter
        return self.__private_var

    def set_private(self, value): # Setter
        self.__private_var = value
```

Example:

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner          # Public
        self._bank = "SBI"          # Protected
        self.__balance = balance    # Private

    def get_balance(self):           # Getter
        return self.__balance

    def deposit(self, amount):       # Setter-like method
        if amount > 0:
            self.__balance += amount

acc = Account("Alice", 1000)
print(acc.owner)                   # Output: Alice
print(acc._bank)                   # Output: SBI
print(acc.get_balance())           # Output: 1000
```

Note on Name Mangling: Python does not strictly enforce private access; it uses name mangling, where `__balance` internally becomes `_Account__balance`. This is a convention that discourages direct access rather than a strict security mechanism.

8.2 Bean Class

Explanation: A Bean class is a plain class containing private fields with public getter and setter methods, and no business logic. This pattern is common in Java but is also applicable in Python.

Syntax:

```
class BeanClassName:
    def __init__(self, field1="", field2=0):
        self.__field1 = field1
        self.__field2 = field2

    def get_field1(self):
        return self.__field1

    def set_field1(self, value):
        self.__field1 = value
```

Example:

```
class PersonBean:
    def __init__(self, name="", age=0):
        self.__name = name
        self.__age = age

    def get_name(self):
        return self.__name

    def get_age(self):
        return self.__age

    def set_name(self, name):
        if isinstance(name, str) and len(name) > 0:
            self.__name = name

    def set_age(self, age):
        if isinstance(age, int) and 0 < age < 150:
            self.__age = age

person = PersonBean()
person.set_name("Alice")
person.set_age(25)
print(person.get_name()) # Output: Alice
```

8.3 Pythonic Encapsulation Using @property

Explanation: The `@property` decorator allows a method to be accessed like an attribute, providing a more Pythonic way to implement getters, setters, and read-only computed values compared to explicit get/set methods.

Syntax:

```
class ClassName:
    def __init__(self, value):
        self._value = value

    @property
    def value(self):
        return self._value

    @value.setter
    def value(self, new_value):
        self._value = new_value
```

Example:

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    @radius.setter
```

```

    def radius(self, value):
        if value <= 0:
            raise ValueError("Radius must be positive")
        self._radius = value

    @property
    def area(self):          # Read-only computed property
        import math
        return math.pi * self._radius ** 2

c = Circle(5)
print(c.radius)    # Output: 5
print(c.area)      # Output: 78.53...
c.radius = 10      # Uses the setter

```

9. Inheritance (is-a Relation)

Explanation: Inheritance is a mechanism through which one class (the child or derived class) acquires the properties and behavior of another class (the parent or base class). It represents an “is-a” relationship, meaning the child class “is a” type of the parent class, and it promotes code reusability.

Syntax:

```

class Parent:
    def method(self):
        pass

class Child(Parent):
    def method(self):    # Overrides parent's method
        pass

```

Example:

```

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):
        return "Bark!"

d = Dog("Buddy")
print(d.name)    # Output: Buddy (inherited attribute)
print(d.speak()) # Output: Bark! (overridden method)

```

9.1 Types of Inheritance

1. Single Level Inheritance

Explanation: In single level inheritance, exactly one child class inherits from exactly one parent class.

Syntax:

```

class Parent:
    pass

class Child(Parent):
    pass

```

Example:

```

class Parent:
    def greet(self):
        return "Hello"

```

```
class Child(Parent):
    def greet(self):
        return "Hi!"
```

2. Multi-Level Inheritance

Explanation: In multi-level inheritance, a class inherits from a child class, which itself inherits from another parent class, forming a chain: Grandparent to Parent to Child.

Syntax:

```
class A:
    pass

class B(A):
    pass

class C(B):
    pass
```

Example:

```
class Grandparent:
    def greet(self):
        return "Hello"

class Parent(Grandparent):
    def greet(self):
        return "Hi"

class Child(Parent):
    def greet(self):
        return "Hey"
```

3. Multiple Inheritance

Explanation: In multiple inheritance, one child class inherits from more than one parent class at the same time. Python resolves conflicts between parent classes using MRO (Method Resolution Order), which determines the order in which classes are searched when looking up a method - in this case, Child, then Father, then Mother, then object.

Syntax:

```
class A:
    pass

class B:
    pass

class C(A, B):
    pass
```

Example:

```
class Father:
    def skill(self):
        return "Driving"

class Mother:
    def skill(self):
        return "Cooking"

class Child(Father, Mother):
    pass

c = Child()
print(c.skill()) # Output: Driving (MRO checks Father first)
```

4. Hierarchical Inheritance

Explanation: In hierarchical inheritance, multiple child classes inherit from a single common parent class.

Syntax:

```
class Parent:
```

```

        pass

    class Child1(Parent):
        pass

    class Child2(Parent):
        pass

```

Example:

```

class Animal:
    def speak(self):
        return "..."

class Dog(Animal):
    def speak(self):
        return "Bark"

class Cat(Animal):
    def speak(self):
        return "Meow"

```

5. Hybrid Inheritance

Explanation: Hybrid inheritance is a combination of two or more of the inheritance types above, used together within the same program.

Syntax:

```

class A:
    pass

class B(A):
    pass

class C:
    pass

class D(B, C):
    pass

```

Example:

```

class A:
    def show(self):
        print("A")

class B(A):
    def display(self):
        print("B")

class C:
    def info(self):
        print("C")

class D(B, C): # Combination of multi-level and multiple inheritance
    pass

d = D()
d.show()
d.display()
d.info()

```

9.2 The super() Function

Explanation: The super() function is used to call a method from the parent class, most commonly used to call the parent's `__init__` method when it has been overridden in the child class.

Syntax:

```

class Child(Parent):
    def __init__(self, parent_args, child_args):
        super().__init__(parent_args)

```

```
self.child_attr = child_args
```

Example:

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # Calls Parent's __init__
        self.age = age
```

9.3 isinstance() and isinstance()

Explanation: `isinstance(object, class)` checks whether an object is an instance of a given class (or its subclasses), returning True or False. `issubclass(class1, class2)` checks whether class1 is a subclass of class2.

Syntax:

```
isinstance(object, class)
issubclass(class1, class2)
```

Example:

```
d = Dog("Buddy")
print(isinstance(d, Dog))      # Output: True
print(isinstance(d, Animal))   # Output: True
print(issubclass(Dog, Animal)) # Output: True
print(issubclass(Dog, object)) # Output: True
```

9.4 has-a Relation

Explanation: Instead of inheriting from another class, a class can contain an object of another class as one of its members. This is called a “has-a” relationship, implemented through Aggregation and Composition.

Aggregation (Weak has-a Relation)

Explanation: Aggregation is a weak has-a relationship, where the contained object can exist independently of the container object. If the container is destroyed, the contained object still survives - there is no ownership. For example, a Library has Books, but the books can exist independently.

Syntax:

```
class Contained:
    pass

class Container:
    def __init__(self, contained_obj): # Passed in from outside
        self.contained_obj = contained_obj
```

Example:

```
class Engine:
    def __init__(self, horsepower):
        self.horsepower = horsepower

class Car:
    def __init__(self, brand, engine):
        self.brand = brand
        self.engine = engine # Engine can exist without Car

engine = Engine(200) # Created independently
car = Car("Toyota", engine)
# If car is deleted, engine still exists independently
```

Composition (Strong has-a Relation)

Explanation: Composition is a strong has-a relationship, where the contained object cannot exist without the container. There is strong

ownership - if the container is destroyed, the contained object is destroyed as well. For example, a House has Rooms, and the rooms cannot exist without the house.

Syntax:

```
class Contained:
    pass

class Container:
    def __init__(self):
        self.contained_obj = Contained()    # Created internally
```

Example:

```
class Address:
    def __init__(self, city, street):
        self.city = city
        self.street = street

class Person:
    def __init__(self, name, city, street):
        self.name = name
        self.address = Address(city, street)    # Created inside Person

p = Person("Alice", "Mumbai", "MG Road")
print(p.address.city)    # Output: Mumbai
```

Aggregation vs Composition Summary: Aggregation represents a weak has-a relationship where the object lifetime is independent and the object is created externally (example: Library and Books), and deleting the container leaves the child intact. Composition represents a strong has-a relationship where the object lifetime is dependent and the object is created internally (example: House and Rooms), and deleting the container deletes the child as well.

10. Polymorphism

Explanation: Polymorphism means “many forms”. It refers to the ability of the same interface, method name, or operator to behave differently depending on the object or data type involved.

10.1 Method Overriding (Run-time Polymorphism)

Explanation: Method overriding occurs when a child class redefines a method that already exists in its parent class, changing its behavior for the child.

Syntax:

```
class Parent:
    def method(self):
        pass

class Child(Parent):
    def method(self):    # Overrides Parent's method
        pass
```

Example:

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return 3.14 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
```

```

    def area(self):
        return self.w * self.h

shapes = [Circle(5), Rectangle(4, 6)]
for s in shapes:
    print(s.area())    # Output: 78.5, then 24 (same method call, different behavior)

```

10.2 Duck Typing

Explanation: Duck typing means that Python does not check the actual type of an object, only whether it has the required behavior (methods). The idea is: “if it walks like a duck and quacks like a duck, it is a duck.”

Syntax:

```

for obj in [obj1, obj2, obj3]:
    obj.common_method()

```

Example:

```

class Dog:
    def speak(self):
        return "Bark"

class Cat:
    def speak(self):
        return "Meow"

class Duck:
    def speak(self):
        return "Quack"

for animal in [Dog(), Cat(), Duck()]:
    print(animal.speak())    # Works for all, regardless of actual class

```

10.3 Operator Overloading

Explanation: Operator overloading allows redefining how standard operators (such as +, ==, <) behave for custom objects, by implementing special “dunder” (double underscore) methods.

Syntax:

```

class ClassName:
    def __add__(self, other):
        # define behavior for +
        pass

```

Example:

```

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):    # Defines behavior of +
        return Point(self.x + other.x, self.y + other.y)

    def __eq__(self, other):    # Defines behavior of ==
        return self.x == other.x and self.y == other.y

    def __repr__(self):        # Defines behavior of repr()
        return f"Point({self.x}, {self.y})"

p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1 + p2)    # Output: Point(4, 6)
print(p1 == p2)   # Output: False

```

Common Dunder Methods for Operator Overloading: `__add__` for +, `__sub__` for -, `__mul__` for *, `__truediv__` for /, `__floordiv__` for //, `__mod__` for %, `__pow__` for **, `__eq__` for ==, `__ne__` for !=, `__lt__` for <, `__le__` for <=, `__gt__` for >, `__ge__` for >=, `__len__` for len(), `__str__` for str(), `__repr__` for repr(), `__contains__` for the in operator, `__getitem__` for

indexing with [], and `__setitem__` for assigning with [] = .

10.4 Function/Method Overloading

Explanation: Python does not support traditional method overloading (defining multiple methods with the same name but different parameters) like some other languages. Instead, this behavior is achieved using default arguments or `*args` / `**kwargs` .

Syntax:

```
class ClassName:
    def method(self, *args):
        # handle variable number of arguments
        pass
```

Example:

```
class Calculator:
    def add(self, *args):
        return sum(args)

c = Calculator()
print(c.add(1, 2))           # Output: 3
print(c.add(1, 2, 3, 4))    # Output: 10
```

11. Abstraction

Explanation: Abstraction hides the internal implementation details of a functionality and shows only the essential features to the user. It focuses on WHAT an object does, rather than HOW it does it. In Python, abstraction is achieved using Abstract Base Classes, provided by the `abc` module.

Syntax:

```
from abc import ABC, abstractmethod

class AbstractClassName(ABC):
    @abstractmethod
    def method_name(self):
        pass
```

Example:

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass

    @abstractmethod
    def stop(self):
        pass

    def display(self):           # Concrete (non-abstract) method
        return "This is a vehicle"

# v = Vehicle() # TypeError: cannot instantiate an abstract class

class Car(Vehicle):
    def start(self):
        return "Car started with key"

    def stop(self):
        return "Car stopped"

c = Car()
print(c.start())               # Output: Car started with key
print(c.display())             # Output: This is a vehicle
```

Note: An abstract class prevents itself from being directly instantiated if it has abstract methods that haven't been implemented by a subclass.

Abstraction vs Encapsulation: Abstraction focuses on hiding complexity and operates at the design level using abstract classes and interfaces, with the goal of showing only essential features. Encapsulation focuses on hiding data and operates at the implementation level using access modifiers, with the goal of protecting the internal state of an object.

12. Singleton Class

Explanation: A Singleton class is a design pattern that restricts a class to only one single instance throughout the entire program. Every subsequent attempt to create an object returns the same existing instance.

Method 1: Using new

Explanation: The `__new__` method controls object creation itself (before `__init__` runs), so it can be overridden to return an already-created instance instead of creating a new one.

Syntax:

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Example:

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self, value=None):
        self.value = value

a = Singleton(10)
b = Singleton(20)
print(a is b)      # Output: True
print(a.value)     # Output: 20 (shared instance, value overwritten)
```

Method 2: Using a Metaclass

Explanation: A metaclass controls how classes themselves are created; overriding `__call__` on a metaclass allows intercepting every instantiation attempt to enforce a single shared instance.

Syntax:

```
class SingletonMeta(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class ClassName(metaclass=SingletonMeta):
    pass
```

Example:

```
class SingletonMeta(type):
    _instances = {}
```

```

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class Database(metaclass=SingletonMeta):
    def __init__(self):
        self.connection = "Connected"

db1 = Database()
db2 = Database()
print(db1 is db2)  # Output: True

```

Method 3: Using a Decorator

Explanation: A decorator function can wrap a class so that it stores and returns the same instance on every subsequent call, achieving the singleton behavior without modifying the class itself.

Syntax:

```

def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class ClassName:
    pass

```

Example:

```

def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class Logger:
    def __init__(self):
        self.logs = []

log1 = Logger()
log2 = Logger()
print(log1 is log2)  # Output: True

```

13. Functions (4 Types) and Inner Functions

Explanation: Python supports four main types of functions, each serving a different purpose, plus inner (nested) functions that support closures.

13.1 Regular (User-Defined Named) Function

Explanation: A regular function is defined using the `def` keyword and is a reusable, named block of code that can accept parameters (positional, keyword, default, `*args`, `**kwargs`) and return a value.

Syntax:

```

def function_name(parameters):
    # function body
    return value

```

Example:

```
def greet(name, greeting="Hello"):
    """Return a greeting message."""
    return f"{greeting}, {name}!"

print(greet("Alice"))      # Output: Hello, Alice!
print(greet("Bob", "Hi"))  # Output: Hi, Bob!
```

**Example (with *args and kwargs):

```
def func(a, b, *args, **kwargs):
    print(f"a={a}, b={b}")
    print(f"args={args}")
    print(f"kwargs={kwargs}")

func(1, 2, 3, 4, x=5, y=6)
# Output:
# a=1, b=2
# args=(3, 4)
# kwargs={'x': 5, 'y': 6}
```

13.2 Lambda (Anonymous) Function

Explanation: A lambda function is a small, nameless function containing only a single expression, defined using the “lambda” keyword. It is commonly used for short operations, often passed to functions like `map()`, `filter()`, and `sorted()`.

Syntax:

```
lambda arguments: expression
```

Example:

```
square = lambda x: x ** 2
add = lambda a, b: a + b
print(square(5))    # Output: 25
print(add(3, 4))    # Output: 7

nums = [1, 2, 3, 4, 5]
evens = list(filter(lambda x: x % 2 == 0, nums))
squared = list(map(lambda x: x**2, nums))
sorted_list = sorted(nums, key=lambda x: -x)
```

13.3 Recursive Function

Explanation: A recursive function is a function that calls itself to solve a problem by breaking it into smaller sub-problems. It must have a base case to stop the recursion, otherwise it will run indefinitely.

Syntax:

```
def function_name(parameters):
    if base_condition:
        return base_value
    return function_name(smaller_parameters)
```

Example:

```
def factorial(n):
    if n <= 1:                # Base case
        return 1
    return n * factorial(n - 1) # Recursive case

print(factorial(5))          # Output: 120

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

for i in range(8):
```

```
print(fibonacci(i), end=" ") # Output: 0 1 1 2 3 5 8 13
```

13.4 Inner (Nested) Function

Explanation: An inner function is a function defined inside another function. It has access to the variables of the enclosing (outer) function's scope, a concept known as a closure.

Syntax:

```
def outer_function(outer_param):
    def inner_function(inner_param):
        return outer_param + inner_param
    return inner_function
```

Example:

```
def outer(x):
    def inner(y):
        return x + y # Accesses x from the outer function's scope
    return inner

add5 = outer(5)
print(add5(3)) # Output: 8
print(add5(10)) # Output: 15
```

Example (using nonlocal to modify enclosing scope variable):

```
def counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

c = counter()
print(c(), c(), c()) # Output: 1 2 3
```

14. Decorators

Explanation: A decorator is a function that takes another function as input, adds some extra functionality to it, and returns a new (modified) function, all without changing the original function's actual code. Decorators use the `@` symbol.

Syntax:

```
def decorator_function(func):
    def wrapper(*args, **kwargs):
        # extra functionality before
        result = func(*args, **kwargs)
        # extra functionality after
        return result
    return wrapper

@decorator_function
def original_function():
    pass
```

Example:

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("--- Before ---")
        result = func(*args, **kwargs)
        print("--- After ---")
        return result
    return wrapper
```

```

@my_decorator
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Alice")
# Output:
# --- Before ---
# Hello, Alice!
# --- After ---

```

14.1 Built-in Decorators

@property — Explanation: Turns a method into a read-only attribute-like accessor. Example:

```

class Circle:
    def __init__(self, r):
        self._r = r

    @property
    def area(self):
        return 3.14 * self._r ** 2

```

@staticmethod — Explanation: Defines a method that does not receive `self` or `cls`, and behaves like a plain function placed inside a class for organizational purposes. Example:

```

class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

```

@classmethod — Explanation: Defines a method that receives `cls` (the class itself) as its first parameter instead of an instance, often used for alternative constructors or accessing class-level data. Example:

```

class Person:
    count = 0

    def __init__(self, name):
        self.name = name
        Person.count += 1

    @classmethod
    def total(cls):
        return cls.count

```

14.2 Decorator with Arguments

Explanation: A decorator that itself accepts arguments requires an extra outer function layer to first receive the arguments, and then return the actual decorator.

Syntax:

```

def decorator_with_args(arg):
    def decorator(func):
        def wrapper(*args, **kwargs):
            # use arg here
            return func(*args, **kwargs)
        return wrapper
    return decorator

```

Example:

```

def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

```

```
@repeat(times=3)
def say_hi():
    print("Hi!")

say_hi() # Prints "Hi!" three times
```

14.3 Chaining Decorators

Explanation: Multiple decorators can be applied to a single function by stacking them; they are applied bottom-up, meaning the decorator closest to the function runs first.

Syntax:

```
@decorator1
@decorator2
def function_name():
    pass
```

Example:

```
def bold(func):
    def wrapper():
        return "<b>" + func() + "</b>"
    return wrapper

def italic(func):
    def wrapper():
        return "<i>" + func() + "</i>"
    return wrapper

@bold
@italic
def hello():
    return "Hello"

print(hello()) # Output: <b><i>Hello</i></b>
```

15. File Handling

Explanation: File handling allows a Python program to create, read, write, and manipulate files stored on disk, using the built-in `open()` function along with various file modes.

15.1 File Opening Modes

Explanation: The mode parameter of `open()` determines how the file is accessed.

r — Read only mode (default); raises an error if the file does not exist. **w** — Write only mode; creates a new file or overwrites existing content. **a** — Append mode; adds new data to the end of the file, creating it if it does not exist. **x** — Create mode; raises an error if the file already exists. **r+** — Allows both reading and writing to the file. **w+** — Allows writing and reading, but truncates (overwrites) existing content. **a+** — Allows appending and reading. **rb**, **wb**, **ab** — Same as above modes but used for binary files.

15.2 File Reading Operations

Explanation: The `with` statement is recommended for opening files since it automatically closes the file afterward, even if an error occurs.

read() — Explanation: Reads and returns the entire content of the file as a single string. Syntax: `file.read()`. Example:

```
with open("file.txt", "r") as f:
    content = f.read()
```

readline() — Explanation: Reads and returns a single line from the file. Syntax: `file.readline()`. Example:

```
with open("file.txt", "r") as f:
    line = f.readline()
```

readlines() — Explanation: Reads all lines of the file and returns them as a list of strings. Syntax: `file.readlines()` . Example:

```
with open("file.txt", "r") as f:
    lines = f.readlines()
```

Iterating a file directly: A file object can be iterated line by line directly in a for loop, which is memory-efficient for large files.

Example:

```
with open("file.txt", "r") as f:
    for line in f:
        print(line.strip())
```

15.3 File Writing Operations

write(string) — Explanation: Writes the given string to the file. Syntax: `file.write(string)` . Example:

```
with open("file.txt", "w") as f:
    f.write("Hello\n")
    f.write("World\n")
```

Appending: Opening a file in “a” mode adds data to the end without erasing existing content.

Example:

```
with open("file.txt", "a") as f:
    f.write("Appended line\n")
```

writelines(list) — Explanation: Writes a list of strings to the file, one after another, without adding newlines automatically. Syntax: `file.writelines(list_of_strings)` . Example:

```
with open("file.txt", "w") as f:
    f.writelines(["line1\n", "line2\n", "line3\n"])
```

15.4 Other File Methods

tell() — Explanation: Returns the current position of the file’s cursor. Syntax: `file.tell()` . Example: `f.tell()` .

seek(n) — Explanation: Moves the file’s cursor to the specified byte position n. Syntax: `file.seek(n)` . Example: `f.seek(0)` moves to the beginning of the file.

flush() — Explanation: Forces any buffered write data to be written to disk immediately. Syntax: `file.flush()` .

close() — Explanation: Closes the opened file, releasing system resources. Syntax: `file.close()` .

15.5 File Paths (os and pathlib modules)

Explanation: The `os` module and the more modern `pathlib` module both provide tools for working with file paths and the filesystem.

Syntax and Example (os module):

```
import os

os.path.exists("file.txt")           # Checks if file exists
os.path.getsize("file.txt")          # Returns file size in bytes
os.listdir(".")                      # Lists files in a directory
os.makedirs("dir1/dir2", exist_ok=True) # Creates nested directories
os.remove("file.txt")                # Deletes a file
os.rename("old.txt", "new.txt")       # Renames a file
```

Syntax and Example (pathlib module):

```
from pathlib import Path

p = Path("file.txt")
p.exists()           # Checks if file exists
p.read_text()        # Reads file content as text
p.write_text("Hello") # Writes text to file
p.parent             # Returns the parent directory
p.suffix              # Returns the file extension
```

15.6 Working with CSV Files

Explanation: The `csv` module provides functionality to read from and write to CSV (Comma Separated Values) files.

Syntax (Writing):

```
import csv
with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(row)
    writer.writerows(list_of_rows)
```

Example (Writing):

```
import csv
with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Age"])
    writer.writerows([["Alice", 22], ["Bob", 25]])
```

Syntax (Reading):

```
with open("data.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

16. Exception Handling

Explanation: Exception handling is a mechanism used to handle runtime errors gracefully so that the program does not crash unexpectedly. It uses the keywords `try`, `except`, `else`, `finally`, and `raise`.

16.1 Basic try-except

Syntax:

```
try:
    # code that may raise an error
except ExceptionType:
    # code to handle the error
```

Example:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

16.2 Multiple except Blocks

Explanation: Multiple except blocks allow handling different types of exceptions differently, with a general `Exception` catch-all as the last block.

Example:

```
try:
    num = int(input("Enter number: "))
    result = 10 / num
except ValueError:
    print("Invalid input!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print(f"Error: {e}")
```

16.3 try-except-else-finally

Explanation: The `else` block runs only if no exception occurred in the try block. The `finally` block always runs, whether an exception occurred or not, and is typically used for cleanup actions.

Syntax:

```
try:
    # code that may raise an error
except ExceptionType:
    # handle error
else:
    # runs if no exception occurred
finally:
    # always runs
```

Example:

```
try:
    f = open("file.txt", "r")
except FileNotFoundError:
    print("File not found!")
else:
    print(f.read()) # Runs only if no exception
finally:
    print("Always runs") # Always executes
```

16.4 Common Exception Types

ValueError — raised when a value has the correct type but an invalid value. **TypeError** — raised when an operation is applied to an object of an inappropriate type. **IndexError** — raised when a sequence index is out of range. **KeyError** — raised when a dictionary key is not found. **FileNotFoundError** — raised when a file or directory does not exist. **AttributeError** — raised when an invalid attribute is accessed. **ImportError** — raised when a module cannot be imported. **ZeroDivisionError** — raised when dividing by zero. **StopIteration** — raised by an iterator when it has no more items. **OverflowError** — raised when the result of an operation is too large to represent.

16.5 Raising Exceptions

Explanation: The `raise` keyword is used to manually trigger an exception when a specific condition occurs.

Syntax:

```
raise ExceptionType("error message")
```

Example:

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    set_age(-5)
except ValueError as e:
    print(e) # Output: Age cannot be negative
```

16.6 Custom Exceptions

Explanation: A custom exception is created by defining a new class that inherits from the built-in `Exception` class, allowing exceptions specific to an application's needs.

Syntax:

```
class CustomError(Exception):
    def __init__(self, message):
        super().__init__(message)
```

Example:

```

class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"Cannot withdraw {amount}. Balance: {balance}")

    def withdraw(balance, amount):
        if amount > balance:
            raise InsufficientFundsError(balance, amount)
        return balance - amount

    try:
        new_bal = withdraw(100, 150)
    except InsufficientFundsError as e:
        print(e)

```

16.7 Exception Chaining

Explanation: Exception chaining, using the `from` keyword, links a new exception to the original one that caused it, preserving the full traceback context for debugging.

Syntax:

```
raise NewException("message") from original_exception
```

Example:

```

try:
    open("file.txt")
except FileNotFoundError as e:
    raise RuntimeError("Config failed") from e

```

17. Higher Order Functions

Explanation: A higher order function is a function that either takes one or more functions as arguments, or returns a function as its result. Python provides three commonly used higher order functions: `map()`, `filter()`, and `reduce()`.

17.1 map(function, iterable)

Explanation: Applies the given function to every element of an iterable, returning an iterator of the transformed results.

Syntax:

```
map(function, iterable)
```

Example:

```

numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, numbers))
print(squared) # Output: [1, 4, 9, 16, 25]

a = [1, 2, 3]
b = [10, 20, 30]
result = list(map(lambda x, y: x + y, a, b)) # map with multiple iterables
print(result) # Output: [11, 22, 33]

```

17.2 filter(function, iterable)

Explanation: Filters the elements of an iterable, keeping only those for which the given function returns True.

Syntax:

```
filter(function, iterable)
```

Example:

```

numbers = [1, 2, 3, 4, 5, 6, 7, 8]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)    # Output: [2, 4, 6, 8]

words = ["hello", "", "world", "", "python"]
non_empty = list(filter(None, words))    # None as function filters out falsy values
print(non_empty)    # Output: ['hello', 'world', 'python']

```

17.3 reduce(function, iterable)

Explanation: Applies the given function cumulatively to the items of an iterable, reducing it to a single value. It is available in the `functools` module.

Syntax:

```

from functools import reduce
reduce(function, iterable)

```

Example:

```

from functools import reduce

numbers = [1, 2, 3, 4, 5]
total = reduce(lambda a, b: a + b, numbers)
print(total)    # Output: 15

product = reduce(lambda a, b: a * b, numbers)
print(product)    # Output: 120

max_val = reduce(lambda a, b: a if a > b else b, numbers)
print(max_val)    # Output: 5

```

Comparing the Three: `map()` transforms each element and returns a new iterable of the same length. `filter()` selects only the matching elements and returns a filtered iterable, possibly shorter than the original. `reduce()` combines all elements into a single final value.

18. Iterators

Explanation: An iterable is any object that can be looped over, such as a list, tuple, or string. An iterator is an object that implements both the `__iter__()` and `__next__()` methods, allowing sequential traversal of elements without exposing the underlying structure.

18.1 Creating a Custom Iterator

Syntax:

```

class IteratorClass:
    def __iter__(self):
        return self

    def __next__(self):
        if condition_met:
            raise StopIteration
        return next_value

```

Example:

```

class CountUp:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.end:
            raise StopIteration

```

```

        value = self.current
        self.current += 1
        return value

for num in CountUp(1, 5):
    print(num, end=" ") # Output: 1 2 3 4 5

```

18.2 Using iter() and next()

Explanation: The `iter()` function returns an iterator from an iterable, and `next()` retrieves the next value from that iterator, raising a `StopIteration` error once exhausted.

Syntax:

```

iterator = iter(iterable)
next(iterator)

```

Example:

```

nums = [10, 20, 30]
itr = iter(nums)
print(next(itr)) # Output: 10
print(next(itr)) # Output: 20
print(next(itr)) # Output: 30
# print(next(itr)) # Would raise StopIteration

```

18.3 How the for Loop Works Internally

Explanation: A `for` loop internally calls `iter()` on the iterable to get an iterator, then repeatedly calls `next()` until a `StopIteration` exception is raised, at which point the loop exits automatically.

Example:

```

# This loop:
for item in iterable:
    print(item)

# is equivalent to:
iterator = iter(iterable)
while True:
    try:
        item = next(iterator)
        print(item)
    except StopIteration:
        break

```

18.4 Infinite Iterator

Explanation: An iterator that never raises `StopIteration` will continue producing values indefinitely; such iterators must be used carefully, typically stopped by an external condition.

Example:

```

class InfiniteCounter:
    def __init__(self):
        self.num = 0

    def __iter__(self):
        return self

    def __next__(self):
        self.num += 1
        return self.num

counter = InfiniteCounter()
print(next(counter)) # Output: 1
print(next(counter)) # Output: 2

```

19. Generators (Function and Expression)

19.1 Generator Function

Explanation: A generator function is a function that uses the `yield` keyword instead of `return`. It produces values lazily, one at a time, pausing its state between each call and resuming from where it left off, which makes it memory efficient.

Syntax:

```
def generator_function():  
    yield value1  
    yield value2
```

Example:

```
def countdown(n):  
    while n > 0:  
        yield n  
        n -= 1  
  
for num in countdown(5):  
    print(num, end=" ") # Output: 5 4 3 2 1  
  
gen = countdown(3)  
print(next(gen)) # Output: 3  
print(next(gen)) # Output: 2  
print(next(gen)) # Output: 1
```

Generator vs Regular Function: A regular function uses the `return` keyword, runs completely to the end, and returns a single value all at once. A generator function uses `yield`, pauses execution at each yield statement, stores only one value in memory at a time, and produces a sequence of values over multiple calls.

19.2 Generator Expression

Explanation: A generator expression is a concise way to create a generator, similar in syntax to a list comprehension but written with parentheses instead of square brackets. It is memory-efficient because it produces values lazily instead of storing them all in memory at once.

Syntax:

```
(expression for item in iterable)
```

Example:

```
nums_list = [x**2 for x in range(1000000)] # List comprehension: builds the full list in memory  
nums_gen = (x**2 for x in range(1000000)) # Generator expression: lazy, memory efficient  
  
print(next(nums_gen)) # Output: 0  
print(next(nums_gen)) # Output: 1  
print(next(nums_gen)) # Output: 4  
  
for val in (x**2 for x in range(5)):  
    print(val, end=" ") # Output: 0 1 4 9 16
```

19.3 Generator Use Cases

Reading a large file line by line:

```
def read_large_file(path):  
    with open(path, "r") as f:  
        for line in f:  
            yield line.strip()
```

Generating an infinite Fibonacci sequence:

```
def fibonacci():  
    a, b = 0, 1
```

```

while True:
    yield a
    a, b = b, a + b

fib = fibonacci()
for _ in range(10):
    print(next(fib), end=" ") # Output: 0 1 1 2 3 5 8 13 21 34

```

Sending values into a generator: The `send()` method resumes a generator and passes a value into it, which is received at the point where the generator was paused with `yield`.

Example:

```

def accumulator():
    total = 0
    while True:
        value = yield total
        total += value

acc = accumulator()
next(acc) # Initializes the generator
print(acc.send(10)) # Output: 10
print(acc.send(20)) # Output: 30
print(acc.send(5)) # Output: 35

```

20. Packing and Unpacking

Explanation: Packing refers to combining multiple values into a single variable, such as a tuple, list, or dictionary. Unpacking refers to extracting values from a collection into separate individual variables.

20.1 Unpacking

Syntax:

```

a, b, c = iterable_with_three_items
first, *rest = iterable

```

Example:

```

a, b, c = 1, 2, 3
x, y, z = [10, 20, 30]

first, *rest = [1, 2, 3, 4, 5]
print(first) # Output: 1
print(rest) # Output: [2, 3, 4, 5]

first, *middle, last = [1, 2, 3, 4, 5]
print(middle) # Output: [2, 3, 4]

(a, b), (c, d) = (1, 2), (3, 4) # Nested unpacking

```

20.2 Packing

Explanation: Values can be packed into a tuple through a function's return statement, or collected into a tuple/dictionary using `*args` / `**kwargs` inside a function definition.

Syntax:

```

def function_name(*args, **kwargs):
    # args is packed into a tuple
    # kwargs is packed into a dictionary

```

Example:

```

def func():
    return 1, 2, 3 # Packed into a tuple

```

```

a = func()          # a = (1, 2, 3)
x, y, z = func()    # Unpacking the returned tuple

def sum_all(*args):
    return sum(args)

result = sum_all(1, 2, 3, 4, 5) # args packs all arguments into a tuple

```

20.3 Packing and Unpacking with Collections

Explanation: The `*` operator unpacks iterables (like lists) into separate positional arguments, and the `**` operator unpacks dictionaries into keyword arguments. `**` can also be used to merge multiple dictionaries.

Example:

```

person = {"name": "Alice", "age": 22}
name, age = person.values() # Unpacking dictionary values
key1, key2 = person.keys()  # Unpacking dictionary keys

def greet(name, age):
    return f"{name} is {age}"

info = {"name": "Alice", "age": 22}
print(greet(**info)) # Output: Alice is 22 (dictionary unpacked as keyword arguments)

d1 = {"a": 1}
d2 = {"b": 2}
merged = {**d1, **d2} # Output: {'a': 1, 'b': 2} (merging dictionaries)

nums = [1, 2, 3]
print(*nums) # Output: 1 2 3 (list unpacked into separate arguments for print)

```

21. Comprehensions

Explanation: A comprehension is a concise, one-line syntax used to create lists, dictionaries, sets, or generators from an existing iterable, optionally applying a condition or transformation.

21.1 List Comprehension

Syntax:

```
[expression for item in iterable if condition]
```

Example:

```

squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]

matrix = [[i*3+j+1 for j in range(3)] for i in range(3)]
# Output: [[1,2,3],[4,5,6],[7,8,9]]

words = ["hello", "WORLD", "python"]
lower = [w.lower() for w in words]

```

21.2 Dictionary Comprehension

Syntax:

```
{key_expression: value_expression for item in iterable}
```

Example:

```

squares = {x: x**2 for x in range(6)}
# Output: {0:0, 1:1, 2:4, 3:9, 4:16, 5:25}

```

```
original = {"a": 1, "b": 2, "c": 3}
swapped = {v: k for k, v in original.items()}
# Output: {1:'a', 2:'b', 3:'c'}

filtered = {k: v for k, v in original.items() if v > 1}
```

21.3 Set Comprehension

Syntax:

```
{expression for item in iterable}
```

Example:

```
unique_lengths = {len(word) for word in ["hi", "hello", "hey"]}
# Output: {2, 3, 5}

evens = {x for x in range(10) if x % 2 == 0}
# Output: {0, 2, 4, 6, 8}
```

21.4 Generator Expression (as a Comprehension)

Syntax:

```
(expression for item in iterable)
```

Example:

```
gen = (x**2 for x in range(1000000)) # Parentheses instead of brackets
print(next(gen)) # Output: 0
print(next(gen)) # Output: 1

squares_list = list(x**2 for x in range(10)) # Converted to a list if needed
```

21.5 Nested Comprehension

Explanation: Comprehensions can be nested to flatten nested lists or transform structures like matrices.

Example (Flattening a nested list):

```
nested = [[1, 2], [3, 4], [5, 6]]
flat = [num for sublist in nested for num in sublist]
# Output: [1, 2, 3, 4, 5, 6]
```

Example (Transposing a matrix):

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
transposed = [[row[i] for row in matrix] for i in range(3)]
```

22. Database Connection (SQLite)

Explanation: Python includes a built-in `sqlite3` module for connecting to and working with SQLite databases, requiring no separate installation.

22.1 Connecting and Creating Tables

Syntax:

```
import sqlite3
connection = sqlite3.connect("database_name.db")
cursor = connection.cursor()
cursor.execute("SQL_QUERY")
```

```
connection.commit()
```

Example:

```
import sqlite3

conn = sqlite3.connect("mydb.db")
cursor = conn.cursor()

cursor.execute('''
    CREATE TABLE IF NOT EXISTS students (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER,
        grade TEXT
    )
''')
```

22.2 INSERT Operations

Explanation: `execute()` runs a single SQL statement, typically using `?` as a placeholder to safely insert parameter values and avoid SQL injection. `executemany()` runs the same statement for a list of parameter sets.

Syntax:

```
cursor.execute("INSERT INTO table (col1, col2) VALUES (?, ?)", (val1, val2))
cursor.executemany("INSERT INTO table (col1, col2) VALUES (?, ?)", list_of_tuples)
connection.commit()
```

Example:

```
cursor.execute(
    "INSERT INTO students (name, age, grade) VALUES (?, ?, ?)",
    ("Alice", 22, "A")
)
conn.commit()

data = [("Bob", 21, "B"), ("Charlie", 23, "A")]
cursor.executemany(
    "INSERT INTO students (name, age, grade) VALUES (?, ?, ?)",
    data
)
conn.commit()
```

22.3 SELECT Queries

Explanation: `fetchone()` retrieves a single row from the result set, while `fetchall()` retrieves all remaining rows as a list of tuples.

Syntax:

```
cursor.execute("SELECT * FROM table WHERE condition")
cursor.fetchone()
cursor.fetchall()
```

Example:

```
cursor.execute("SELECT * FROM students WHERE id = ?", (1,))
row = cursor.fetchone()
print(row)  # Output: (1, 'Alice', 22, 'A')

cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
for row in rows:
    print(row)

cursor.execute("SELECT * FROM students WHERE age > ?", (21,))
for row in cursor.fetchall():
    print(row)
```

22.4 UPDATE and DELETE Operations

Syntax:

```
cursor.execute("UPDATE table SET column = ? WHERE condition = ?", (new_val, cond_val))
cursor.execute("DELETE FROM table WHERE condition = ?", (cond_val,))
connection.commit()
```

Example:

```
cursor.execute(
    "UPDATE students SET grade = ? WHERE name = ?",
    ("A+", "Alice")
)
conn.commit()

cursor.execute("DELETE FROM students WHERE id = ?", (3,))
conn.commit()
```

22.5 Using a Context Manager

Explanation: Using `with` when connecting to a database ensures the connection is properly managed and resources are released after use.

Example:

```
import sqlite3

with sqlite3.connect("database.db") as conn:
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM students")
    for row in cursor.fetchall():
        print(row)
```

22.6 Using Row Factory

Explanation: Setting `conn.row_factory = sqlite3.Row` allows accessing query result columns by name instead of only by numeric index, making code more readable.

Example:

```
conn = sqlite3.connect("database.db")
conn.row_factory = sqlite3.Row
cursor = conn.cursor()
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row["name"], row["age"])  # Access by column name
```

Security Note: Always use parameterized queries (the `?` placeholder) instead of directly formatting values into an SQL string, to prevent SQL injection attacks.

23. JSON and Pickle (Serialization and Deserialization)

Explanation: Serialization (also called encoding) is the process of converting a Python object into a format suitable for storage or transmission. Deserialization (also called decoding) is the reverse process, converting stored or transmitted data back into a Python object.

23.1 JSON

Explanation: JSON (JavaScript Object Notation) is a lightweight, text-based, human-readable, and language-independent data format, commonly used in web APIs. It maps naturally to Python dictionaries and lists.

json.dumps(obj) — Explanation: Serializes a Python object into a JSON-formatted string. Syntax: `json.dumps(obj, indent=n)`.
Example:

```
import json
data = {"name": "Alice", "age": 22, "scores": [90, 85, 92], "active": True}
json_str = json.dumps(data, indent=4)
print(json_str)
```

json.loads(string) — Explanation: Deserializes a JSON-formatted string back into a Python object. Syntax: `json.loads(json_string)`. Example:

```
parsed = json.loads(json_str)
print(parsed["name"]) # Output: Alice
```

json.dump(obj, file) — Explanation: Serializes a Python object directly into a file as JSON. Syntax: `json.dump(obj, file, indent=n)`. Example:

```
with open("data.json", "w") as f:
    json.dump(data, f, indent=4)
```

json.load(file) — Explanation: Deserializes JSON data directly from a file into a Python object. Syntax: `json.load(file)`. Example:

```
with open("data.json", "r") as f:
    loaded = json.load(f)
print(type(loaded)) # Output: <class 'dict'>
```

23.2 JSON Serialization of Custom Objects

Explanation: Since JSON does not natively understand custom Python classes, a custom encoder function (passed via the `default` parameter) is needed to convert custom objects into JSON-serializable dictionaries, and an `object_hook` function can convert them back during deserialization.

Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

def person_encoder(obj):
    if isinstance(obj, Person):
        return {"name": obj.name, "age": obj.age}
    raise TypeError(f"Object of type {type(obj)} is not JSON serializable")

p = Person("Alice", 22)
json_str = json.dumps(p, default=person_encoder, indent=2)

def person_decoder(dct):
    if "name" in dct and "age" in dct:
        return Person(dct["name"], dct["age"])
    return dct

person = json.loads(json_str, object_hook=person_decoder)
```

23.3 Pickle

Explanation: Pickle is a Python module that serializes and deserializes Python objects into a binary byte stream. Unlike JSON, pickle is Python-specific, can serialize almost any complex Python object directly, but is not safe to use with untrusted data (loading a malicious pickle file can execute arbitrary code).

pickle.dump(obj, file) — Explanation: Serializes a Python object and writes it to a file in binary format. Syntax: `pickle.dump(obj, file)`. Example:

```
import pickle
data = {"name": "Alice", "scores": [90, 85]}
with open("data.pkl", "wb") as f:
    pickle.dump(data, f)
```

pickle.load(file) — Explanation: Deserializes binary data from a file back into a Python object. Syntax: `pickle.load(file)`. Example:

```
with open("data.pkl", "rb") as f:
    loaded = pickle.load(f)
```

```
print(loaded)
```

pickle.dumps(obj) — Explanation: Serializes a Python object directly into a bytes object (without writing to a file). Syntax: `pickle.dumps(obj)`.

pickle.loads(bytes) — Explanation: Deserializes a bytes object directly back into a Python object. Syntax: `pickle.loads(bytes_obj)`.

JSON vs Pickle: JSON produces a human-readable text format, is not Python-specific (so it works across languages), needs a custom encoder for custom objects, and is safe to use with untrusted data. Pickle produces a binary format that is not human-readable, is Python-specific, supports custom objects directly without extra work, but is unsafe to load from untrusted sources, and typically produces smaller file sizes than JSON.

24. Regular Expressions (re module)

Explanation: Regular expressions (regex) are patterns used to match character combinations within strings. Python's built-in `re` module provides full regex support.

24.1 Core Regex Functions

re.search(pattern, string) — Explanation: Searches the entire string and returns a match object for the first location where the pattern matches, or None if no match is found. Syntax: `re.search(pattern, string)`. Example:

```
import re
match = re.search(r"\d+", "abc123def")
print(match.group()) # Output: 123
```

re.match(pattern, string) — Explanation: Checks for a match only at the very beginning of the string. Syntax: `re.match(pattern, string)`. Example:

```
match = re.match(r"\d+", "123abc")
print(match.group()) # Output: 123
```

re.findall(pattern, string) — Explanation: Returns a list of all non-overlapping matches of the pattern in the string. Syntax: `re.findall(pattern, string)`. Example:

```
matches = re.findall(r"\d+", "a1b2c3")
print(matches) # Output: ['1', '2', '3']
```

re.finditer(pattern, string) — Explanation: Returns an iterator yielding match objects for all non-overlapping matches, useful when you need match positions as well as values. Syntax: `re.finditer(pattern, string)`. Example:

```
for m in re.finditer(r"\d+", "a1b2c3"):
    print(m.group())
```

re.sub(pattern, replacement, string) — Explanation: Replaces all occurrences of the pattern in the string with the replacement string, returning the modified string. Syntax: `re.sub(pattern, replacement, string)`. Example:

```
result = re.sub(r"\d+", "X", "a1b2c3")
print(result) # Output: aXbXcX
```

re.split(pattern, string) — Explanation: Splits the string wherever the pattern matches, returning a list of substrings. Syntax: `re.split(pattern, string)`. Example:

```
parts = re.split(r"[;,]", "hello;world;python rocks")
print(parts) # Output: ['hello', 'world', 'python', 'rocks']
```

24.2 Common Regex Pattern Symbols

`.` matches any character except a newline. `\d` matches any digit (0-9). `\D` matches any non-digit character. `\w` matches any word character (letters, digits, underscore). `\W` matches any non-word character. `\s` matches any whitespace character. `\S` matches any non-whitespace character. `^` matches the start of the string. `$` matches the end of the string. `*` matches zero or more occurrences of the preceding element. `+` matches one or more occurrences. `?` matches zero or one occurrence. `{n}` matches exactly n occurrences. `{n,m}` matches between n and m occurrences. `[abc]` matches any one character inside the brackets (a character set). `[^abc]` matches any character NOT inside the brackets (negated set). `|` acts as a logical OR between patterns. `()` groups part of a pattern

together.

24.3 Practical Regex Examples

Email validation pattern:

```
pattern = r"[\w.-]+@[ \w.-]+\w+\"
emails = re.findall(pattern, text)
```

Phone number pattern:

```
pattern = r"\d{3}-\d{3}-\d{4}"
```

Strong password pattern (8+ characters, upper, lower, digit, special character):

```
pattern = r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$"
```

Extracting HTML tags:

```
pattern = r"<(\w+)>(.*?)</\1>"
matches = re.findall(pattern, "<b>bold</b>", re.DOTALL)
```

24.4 Regex Flags

Explanation: Flags modify how a regex pattern is matched. `re.IGNORECASE` (or `re.I`) makes the match case-insensitive.

`re.MULTILINE` (or `re.M`) makes `^` and `$` match the start/end of each line rather than the whole string. `re.DOTALL` (or `re.S`) makes `.` also match newline characters. `re.VERBOSE` (or `re.X`) allows whitespace and comments within the pattern for readability.

Example:

```
re.findall(r"hello", "HELLO hello", re.IGNORECASE)
# Output: ['HELLO', 'hello']
```

24.5 Groups and Named Groups

Explanation: Parentheses in a pattern create groups, which can be accessed individually from a match object using `group(n)` or all together using `groups()`. Named groups, defined with `(?P<name>...)`, can be accessed by name using `group("name")` or as a dictionary using `groupdict()`.

Example:

```
match = re.search(r"(\d{3})-(\d{3})-(\d{4})", "Call 123-456-7890")
print(match.group(0)) # Output: 123-456-7890 (full match)
print(match.group(1)) # Output: 123
print(match.group(2)) # Output: 456
print(match.groups()) # Output: ('123', '456', '7890')

match = re.search(r"(?P<area>\d{3})-(?P<prefix>\d{3})-(?P<num>\d{4})", "123-456-7890")
print(match.group("area")) # Output: 123
print(match.groupdict()) # Output: {'area': '123', 'prefix': '456', 'num': '7890'}
```

Non-capturing group — Explanation: A group written as `(?:...)` groups part of the pattern without capturing it as a separate result, useful when grouping is needed only for applying an operator like `|`.

Example:

```
re.findall(r"(?:http|https)://[\w.]+", "Visit https://google.com")
```

25. Additional Important Concepts

25.1 Walrus Operator (Python 3.8+)

Explanation: The walrus operator `:=` assigns a value to a variable as part of a larger expression, allowing assignment and evaluation in

a single line, commonly used inside `if` or `while` conditions.

Syntax:

```
if (variable := expression) condition:
    # use variable
```

Example:

```
data = [1, 2, 3, 4, 5, 6]
if (n := len(data)) > 5:
    print(f"List is too long ({n} elements)")
```

25.2 Advanced f-strings

Explanation: f-strings support several advanced formatting options, including displaying both the variable name and its value using `=`, controlling decimal places, adding thousands separators, showing percentages, and formatting numbers in binary or hexadecimal form.

Example:

```
name = "Alice"
age = 22
print(f"name={name}, {age=}")          # Output: name='Alice', age=22
print(f"{3.14159:.2f}")               # Output: 3.14
print(f"{1000000:,.}")                # Output: 1,000,000
print(f"{0.5:%}")                     # Output: 50.000000%
print(f"{42:010b}")                   # Output: 0000101010
print(f"{255:#x}")                     # Output: 0xff
```

25.3 Data Classes (Python 3.7+)

Explanation: The `@dataclass` decorator, from the `dataclasses` module, automatically generates common special methods for a class, such as `__init__`, `__repr__`, and `__eq__`, reducing boilerplate code for classes that mainly store data.

Syntax:

```
from dataclasses import dataclass

@dataclass
class ClassName:
    field1: type
    field2: type
```

Example:

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

    def distance(self, other):
        return ((self.x - other.x)**2 + (self.y - other.y)**2)**0.5

p1 = Point(1, 2)
p2 = Point(4, 6)
print(p1)          # Output: Point(x=1, y=2)
print(p1 == p2)    # Output: False (auto-generated __eq__)
```

25.4 Context Managers

Explanation: A context manager manages setup and teardown logic around a block of code using the `with` statement. The `@contextmanager` decorator, from the `contextlib` module, allows creating a context manager using a generator function, where code before `yield` runs on entry and code after `yield` runs on exit.

Syntax:

```
from contextlib import contextmanager
```

```
@contextmanager
def manager_name():
    # setup code
    yield
    # teardown code
```

Example:

```
from contextlib import contextmanager
import time

@contextmanager
def timer():
    start = time.time()
    yield
    print(f"Time: {time.time() - start:.2f}s")

with timer():
    sum(range(1000000))
```

25.5 enumerate() and zip() in Loops

Explanation: `enumerate()` provides both the index and value while looping over an iterable, optionally starting the index from a custom value. `zip()` allows looping over multiple iterables in parallel, pairing corresponding elements together.

Example:

```
fruits = ["apple", "banana", "cherry"]
for i, fruit in enumerate(fruits, start=1):
    print(f"{i}. {fruit}")

names = ["Alice", "Bob"]
grades = ["A", "B"]
for name, grade in zip(names, grades):
    print(f"{name}: {grade}")
```

25.6 Collections Module

Explanation: The `collections` module provides specialized container data types that extend Python's built-in containers.

Counter — Explanation: Counts the occurrences of elements in an iterable and provides a `most_common(n)` method to get the n most frequent elements. Example:

```
from collections import Counter
c = Counter("mississippi")
print(c.most_common(3)) # Output: [('i', 4), ('s', 4), ('p', 2)]
```

defaultdict — Explanation: A dictionary subclass that provides a default value automatically for a missing key, instead of raising a `KeyError`. Example:

```
from collections import defaultdict
dd = defaultdict(list)
for k, v in [("a", 1), ("b", 2), ("a", 3)]:
    dd[k].append(v)
# Output: {'a': [1, 3], 'b': [2]}
```

namedtuple — Explanation: Creates a tuple subclass with named fields, allowing access to elements by name instead of only by index. Example:

```
from collections import namedtuple
Point = namedtuple("Point", "x y")
p = Point(1, 2)
print(p.x, p.y) # Output: 1 2
```

deque — Explanation: A double-ended queue that allows fast appending and popping from both ends, more efficient than a list for this purpose. Example:

```
from collections import deque
```

```
d = deque([1, 2, 3])
d.appendleft(0)
d.append(4)
print(d)    # Output: deque([0, 1, 2, 3, 4])
```

25.7 Itertools Module

Explanation: The `itertools` module provides functions for creating efficient iterators for looping, including infinite iterators and combinatorial generators.

`itertools.count(start, step)` — Explanation: Creates an infinite iterator that counts up from a starting value by a given step. Example:

```
import itertools
for i in itertools.count(10, 2):    # 10, 12, 14, ...
    if i > 20:
        break
```

`itertools.permutations(iterable, r)` — Explanation: Returns all possible orderings of length `r` from the given iterable. Example:

```
print(list(itertools.permutations([1, 2, 3], 2)))
# Output: [(1,2), (1,3), (2,1), (2,3), (3,1), (3,2)]
```

`itertools.combinations(iterable, r)` — Explanation: Returns all possible combinations of length `r` from the given iterable, without regard to order. Example:

```
print(list(itertools.combinations([1, 2, 3], 2)))
# Output: [(1,2), (1,3), (2,3)]
```

End of Notes
